Deliverable 1

Group 7

Github:
https://github.com/SarahAsghar/EECS3311-YorkU-Lab-Equipment-Reservation-Platform-Group19

Part 1:

Task 1: Use Case Diagrams:

Completed by: Seyed Parsa Shoushtari - 219615574

Diagram 1: Account Setup & Access

Actors: User, Head Lab Coordinator

Use cases: Register Account, Approve University-Affiliated Account, Provide ID/Certification Info, Log In

This diagram shows how users register and access the system, including approval of university-affiliated accounts by the Head Lab Coordinator (Req1, Req2, Req8).

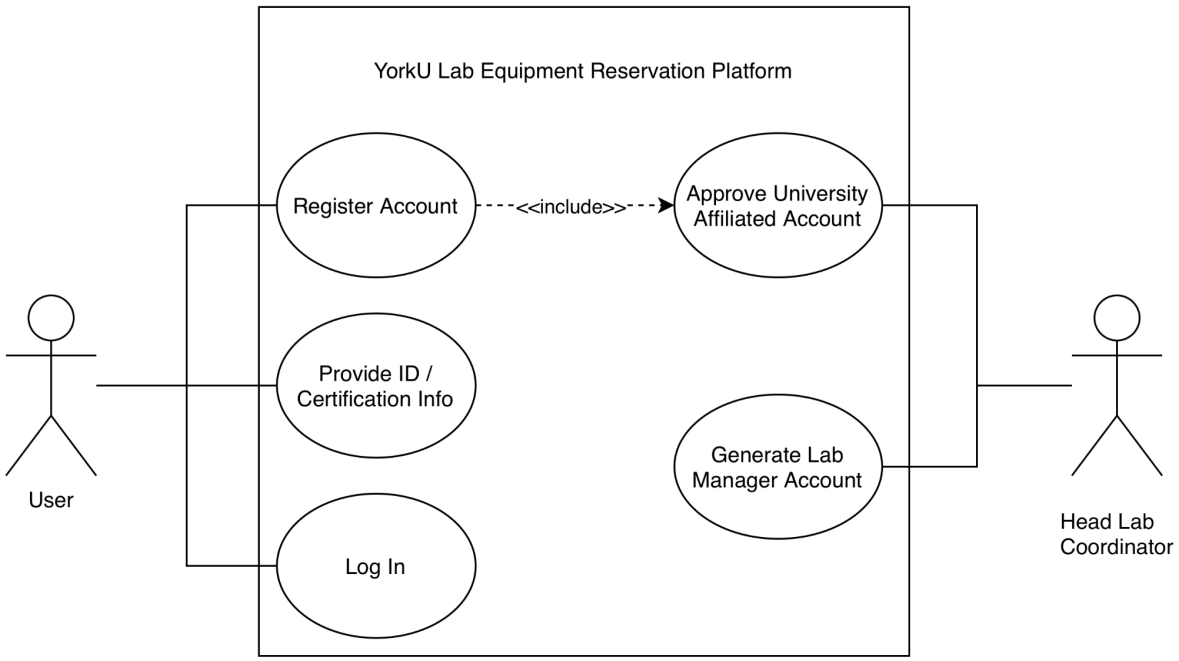


Diagram 2: Reservation & Payment Lifecycle

Actors: User, Payment Service, Equipment Sensor System

Use cases: View Equipment Info/Availability, Reserve Equipment, Pay Deposit, Modify Reservation, Cancel Reservation, Extend Reservation, Send Usage/Status Updates, Handle No-Show Deposit Forfeit

This diagram illustrates the process through which a user views equipment availability, reserves equipment, and manages an existing reservation. It also captures mandatory deposit payment, conditional deposit forfeiture in the event of a no-show, and reservation extension when equipment is available. External systems, including the payment service and equipment sensor system, support payment processing and usage/status tracking. (Req3, Req4, Req5, Req7, Req8, Req9, Req10)

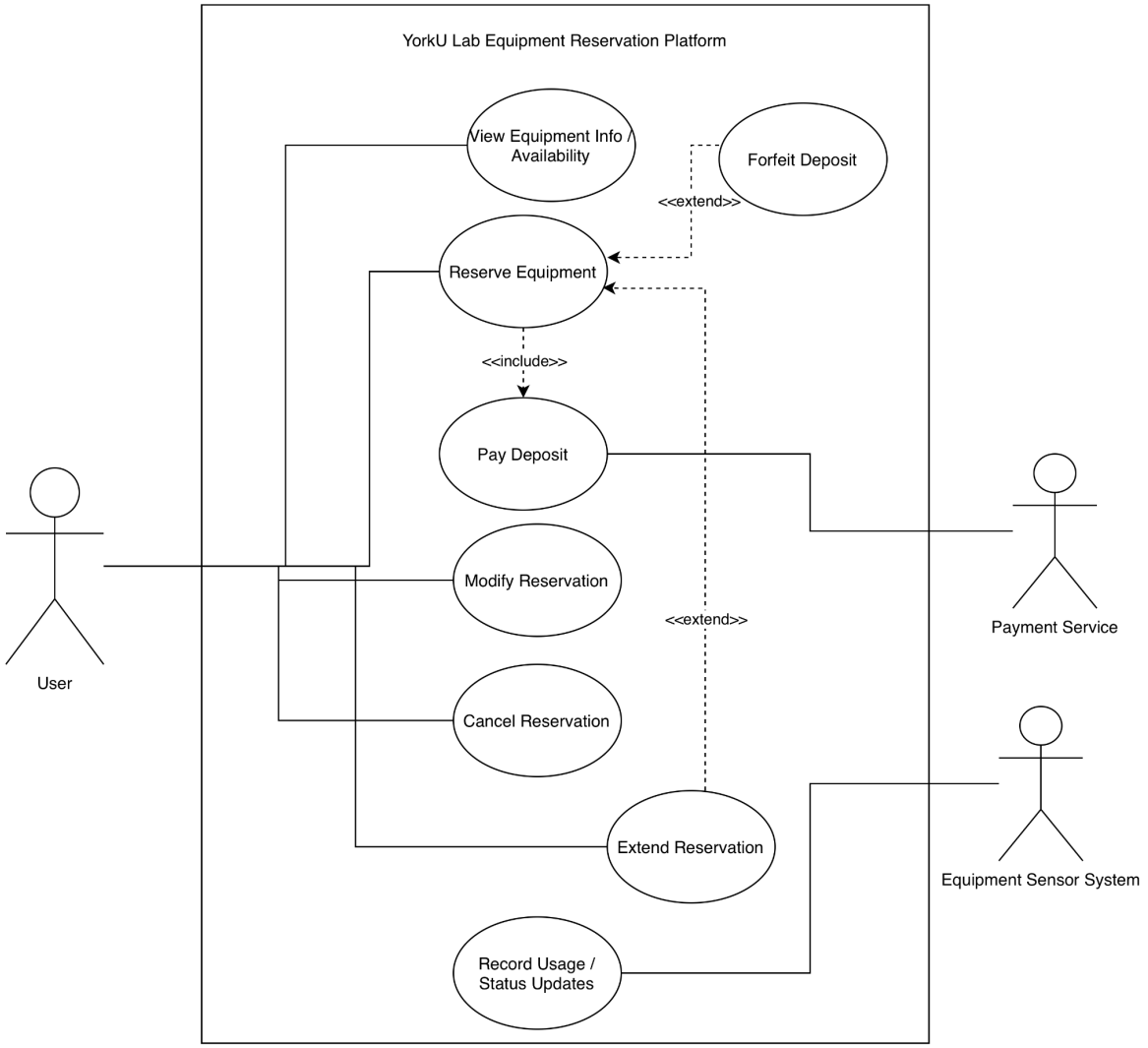
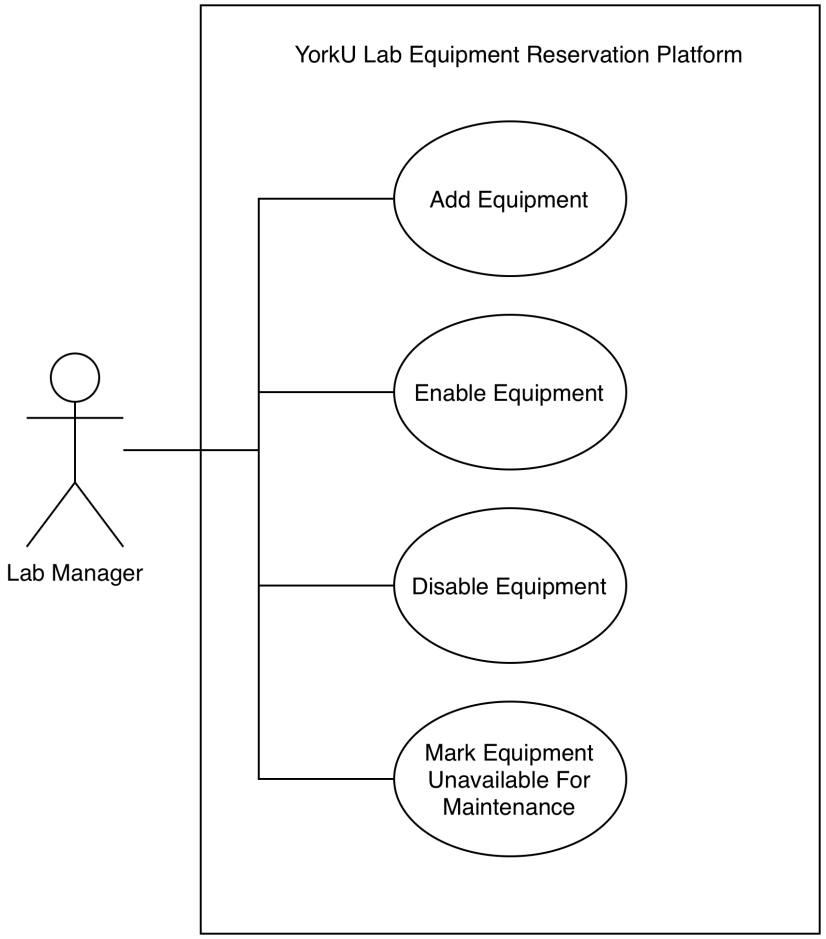


Diagram 3: Equipment Administration

Actors: Lab Manager

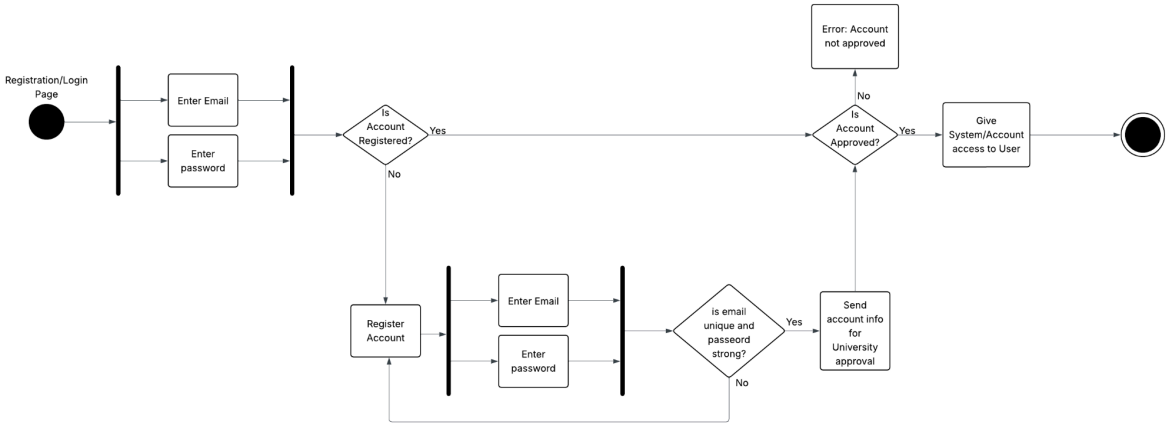
Use cases: Add Equipment, Enable Equipment, Disable Equipment, Mark Unavailable for Maintenance

This diagram shows how the Lab Manager manages the equipment inventory by adding equipment and controlling its availability. The Lab Manager can enable or disable equipment for booking and mark equipment as unavailable when maintenance is required. (Req6, Req7)

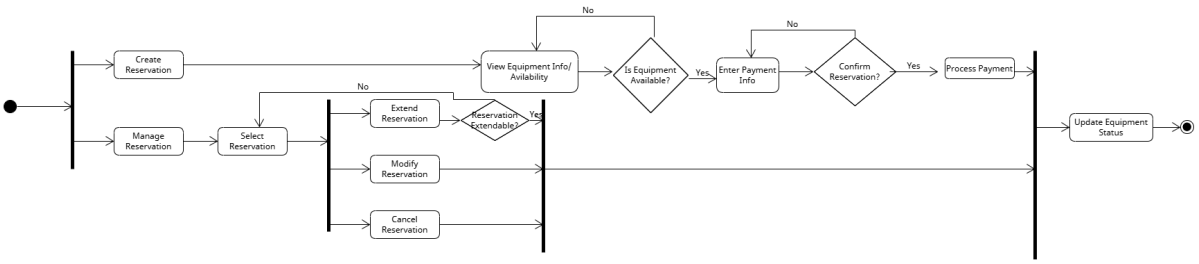


Task 2: Activity Diagram: Doluwamu Olubiremi - 218801563

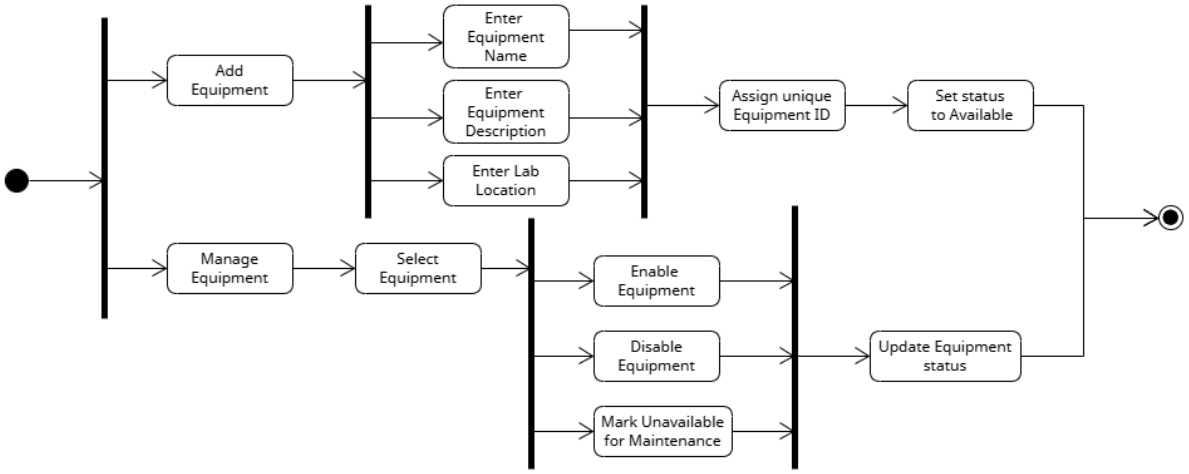
Activity Diagram for Use Case Diagram 1



Activity Diagram for Use Case Diagram 2

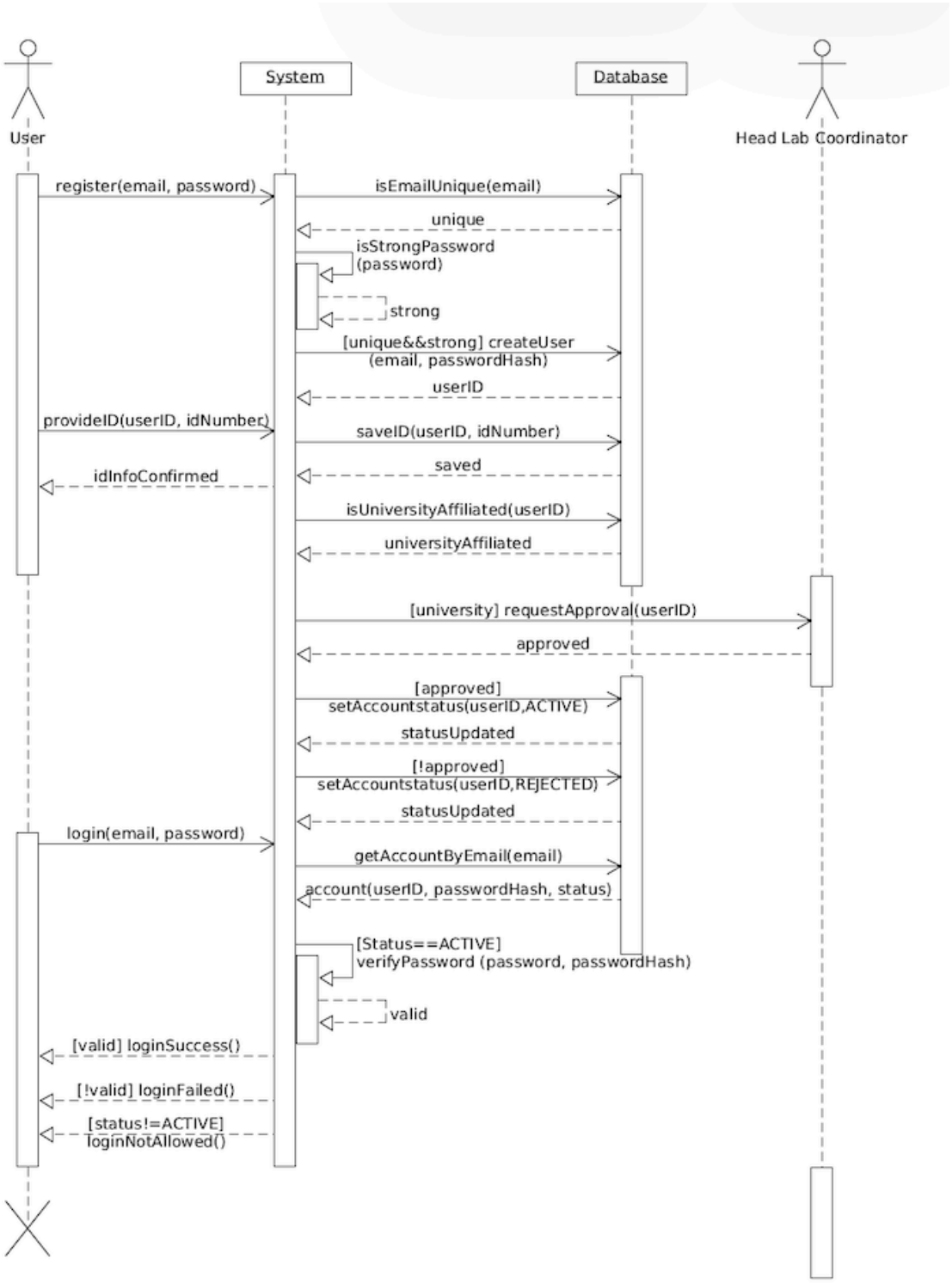


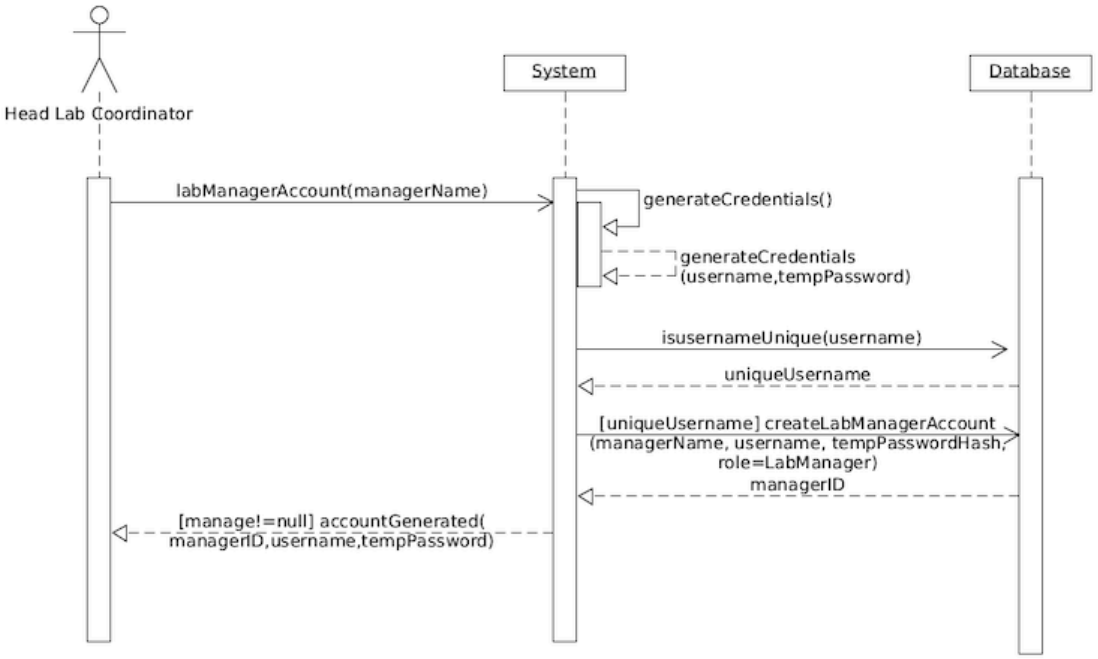
Activity Diagram for Use Case Diagram 3



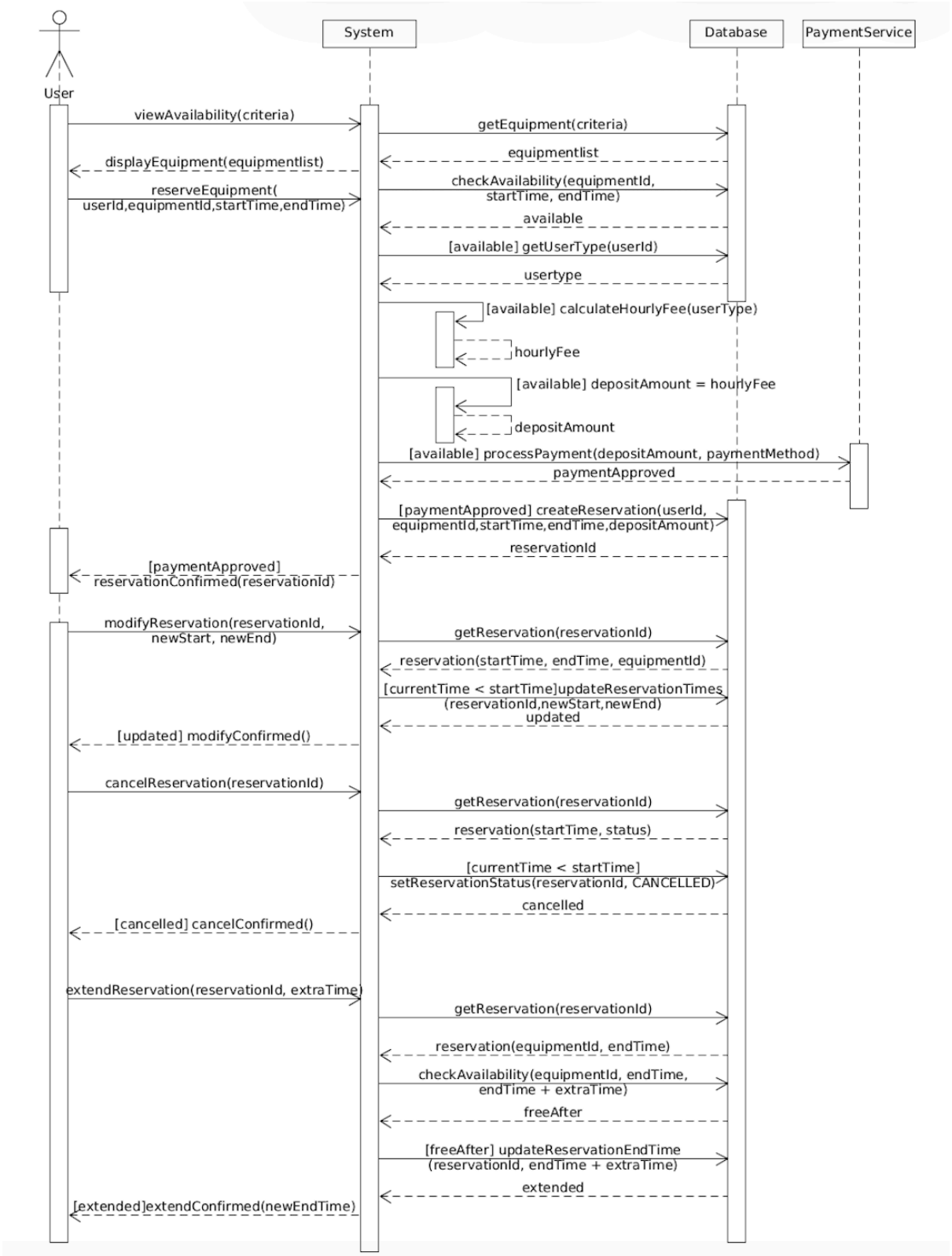
Task 3: Sequence Diagram: Sadia Jahan Shifa - 219302744

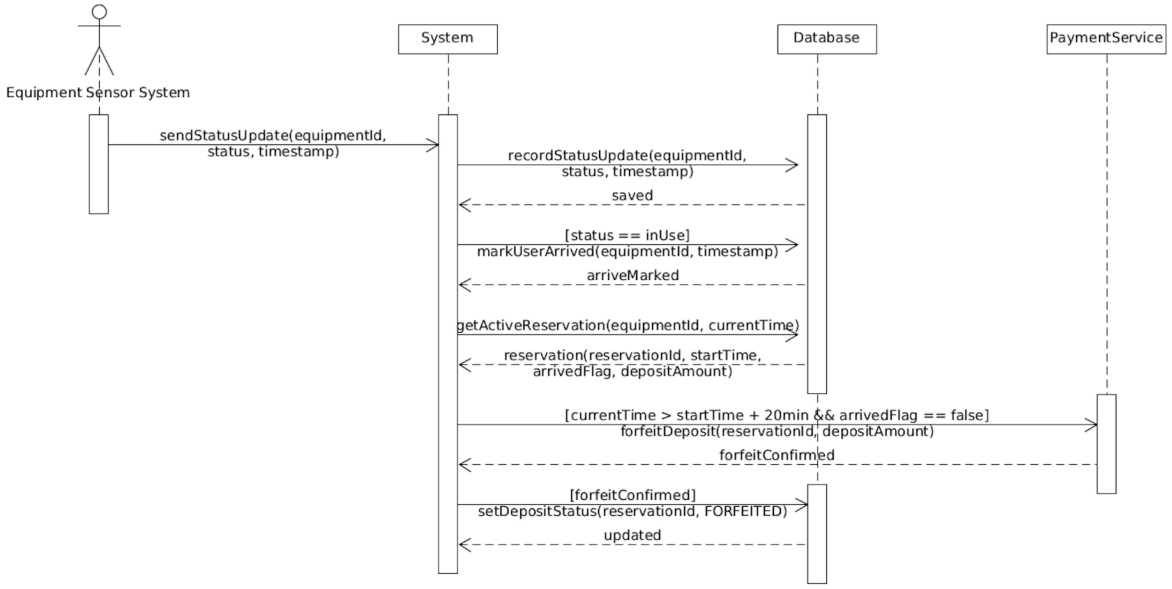
Sequence Diagram for the use case diagram 1:



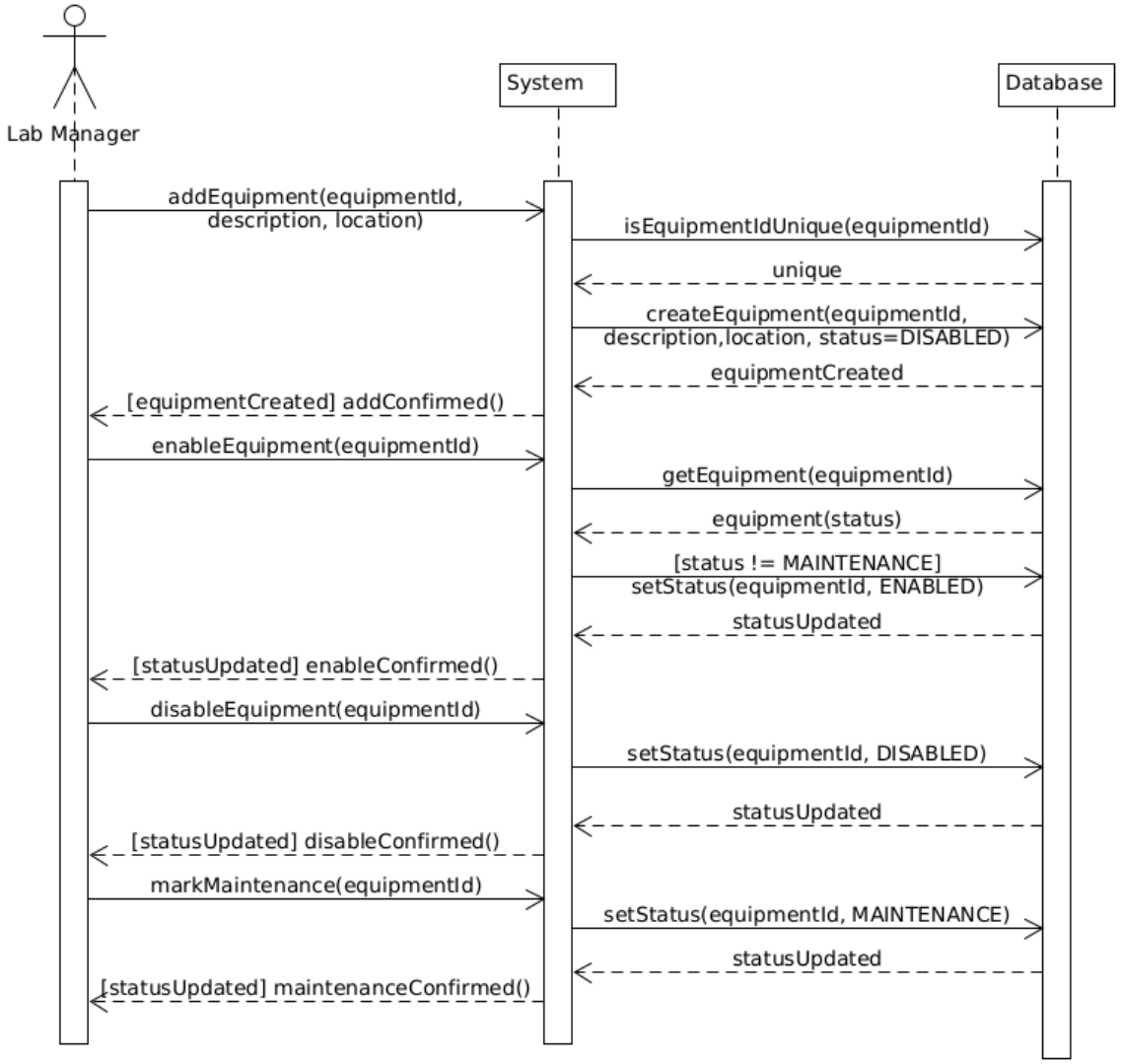

Head Lab Coordinator
System
Database
labManagerAccount(managerName)
generateCredentials()
generateCredentials
(username,tempPassword)
isusernameUnique(username)
uniqueUsername
[uniqueUsername] createLabManagerAccount
(managerName, username, tempPasswordHash,
role=LabManager)
managerID
[manage!=null] accountGenerated(
managerID,username,tempPassword)

Sequence diagram for use case diagram 2:

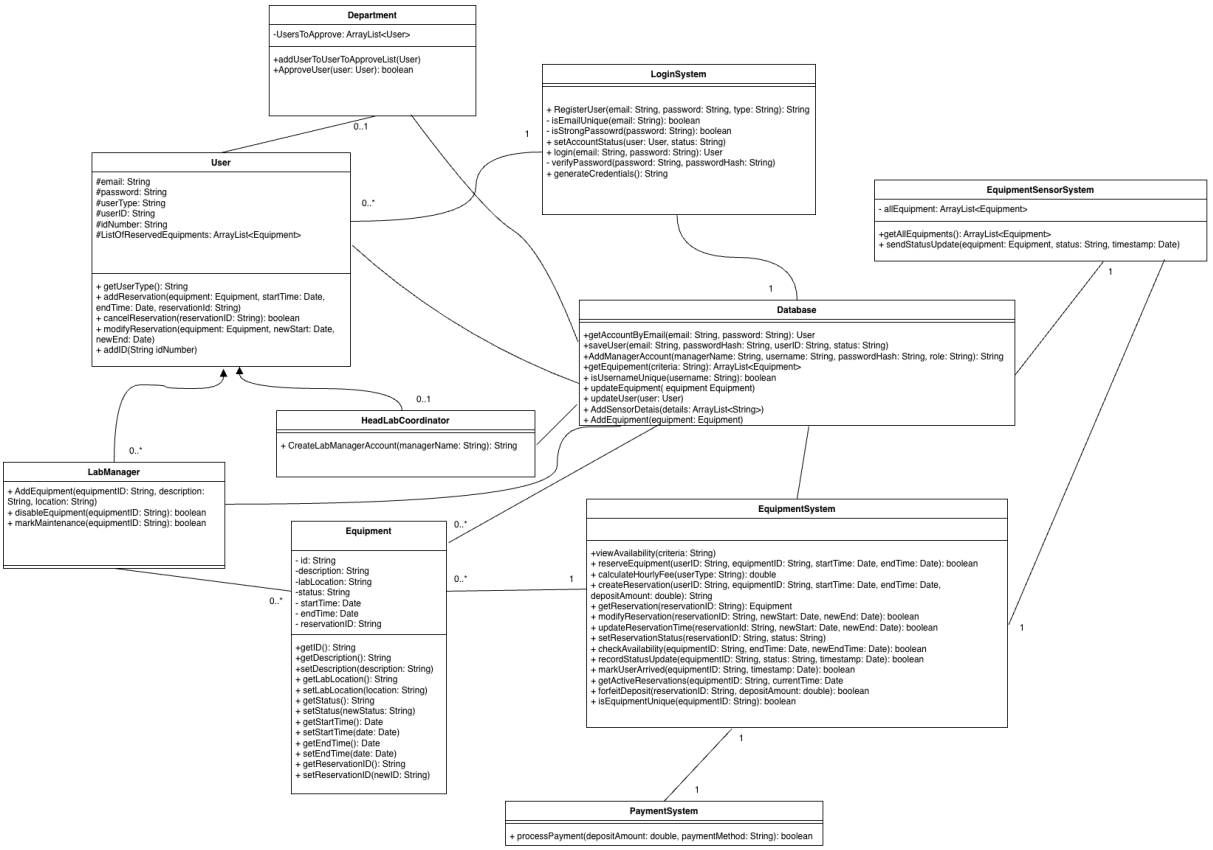



Equipment Sensor System
System
Database
PaymentService
sendStatusUpdate(equipmentId, status, timestamp)
recordStatusUpdate(equipmentId, status, timestamp)
saved
[status == inUse]
markUserArrived(equipmentId, timestamp)
arriveMarked
getActiveReservation(equipmentId, currentTime)
reservation(reservationId, startTime, arrivedFlag, depositAmount)
[currentTime > startTime + 20min && arrivedFlag == false]
forfeitDeposit(reservationId, depositAmount)
forfeitConfirmed
[forfeitConfirmed]
setDepositStatus(reservationId, FORFEITED)
updated

Sequence diagram for use case diagram 3:



Part 2: Class Diagram: Sarah Asghar (219428127)



Part 3: AI Use Report

Part III – Report on AI-Assistant Adaptation

Author: Nabeela Ansari

Task 1 – Role of the AI Assistant

Throughout Part I (Requirements Modeling) and Part II (Design), AI tools were used as support systems for idea refinement, requirement verification, and structural validation. The tools were not used to directly produce final diagrams, but rather to assist with conceptual clarity and completeness.

The AI tools used were:

- ChatGPT-4
- ChatGPT-5
- DeepSeek (for exploratory class diagram generation)

Each task is discussed below.

Part I – Task 1: Use Case Diagram

AI Model Used

ChatGPT-4

Prompt Used

> "Based on these system requirements (Req1–Req10), what potential system actors can be identified, and are any requirements not covered by typical reservation system actors?"

AI Output (Summary)

The AI suggested:

- User
- Head Lab Coordinator
- Lab Manager
- Payment Service
- Equipment Sensor System

It also recommended modeling user types (Student, Faculty, Researcher, Guest) as separate actors.

My Understanding of the Output

The AI was helpful in:

- Identifying external systems (Payment Service and Sensor System)
- Highlighting privilege differences between Head Coordinator and Lab Manager

However, it incorrectly treated Student, Faculty, Researcher, and Guest as separate actors.

According to Req1:

> "User types include students, faculty, researchers, and guests."

These are internal role distinctions, not separate actors interacting differently with the system.

Adjustment Made

We consolidated them into one "User" actor and represented the types using an enumeration (UserType) in the class diagram instead.

This improved UML abstraction and avoided unnecessary actor duplication.

Part I – Task 2: Activity Diagrams

AI Model Used

ChatGPT-5

Prompt Used

> "For a reservation lifecycle, what decision points should appear in a UML activity diagram?"

AI Output (Summary)

The AI suggested decision branches for:

- Equipment availability
- Payment approval
- Reservation extension availability

However, it did not explicitly include:

- The 20-minute no-show deposit forfeiture rule (Req4).

My Understanding

The AI helped structure logical flow and error paths, but it omitted a critical timing condition from Req4.

Adjustment Made

We added:

- A time-based decision node checking:
    - If arrivalTime > startTime + 20 minutes
    - Then deposit is forfeited

This ensured requirement completeness.

Part I – Task 3: Sequence Diagrams

AI Model Used

ChatGPT-5

Prompt Used

> "In a UML sequence diagram for equipment reservation, which objects should be included?"

AI Output (Summary)

Suggested objects:

- User
- System
- Database
- PaymentService
- Sensor

This was largely correct.

However, the AI initially treated PaymentService as an internal system object rather than an external actor.

Adjustment Made

We corrected this to follow UML conventions:

- External systems must be modeled as actors.
- The internal system coordinates interactions.

This correction improved UML compliance.

Part II – Class Diagram Design

AI Models Used

ChatGPT-5
DeepSeek

DeepSeek Prompt Used

> "Give me a class diagram for these requirements: [Req1–Req10]"

DeepSeek Output (Summary)

DeepSeek produced:

- A User base class
- HeadLabCoordinator and LabManager inheriting from User
- Reservation, Equipment, Payment, Department, Sensor classes
- Enumerations for status types

It also suggested:

- One-to-one Reservation-Payment relationship
- Equipment monitored by Sensor
- Department approval linked to User

Why DeepSeek's Design Was Not Fully Followed

Although the structure was comprehensive, several issues were identified:

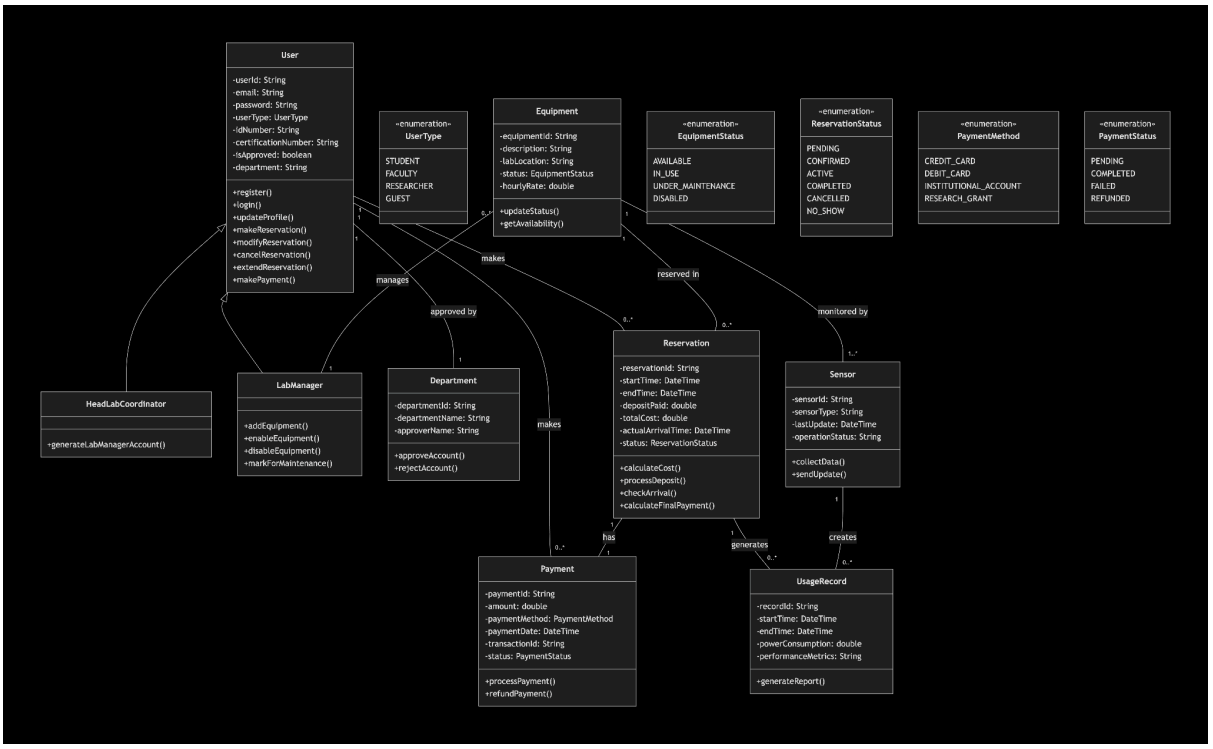
1. It tightly coupled deposit logic inside Payment without representing reservation lifecycle states clearly.

2. It did not clearly enforce multiplicity rules (e.g., one equipment can have many reservations over time).

3. It suggested inheritance that increased complexity unnecessarily.

We used it only as a brainstorming baseline and redesigned the class relationships manually.

The final output was this:



Task 2 – Arguments and Resolution

Several disagreements occurred between AI suggestions and our interpretation of the requirements.

Disagreement 1 – Separate Actors for User Types

AI Suggested:

- Student, Faculty, Researcher, Guest as separate actors.

Problem:

- Req1 defines them as user types, not separate interaction roles.

Resolution:

- Created UserType enumeration instead.
- Kept one User actor in use case diagrams.
- Simplified class structure.

Disagreement 2 – Deposit Handling

AI Suggested:

- Deposit processed but did not model forfeiture condition clearly.

Problem:

- Req4 explicitly requires time-based forfeiture logic.

Resolution:

- Added:
    - actualArrivalTime attribute in Reservation
    - checkArrival() method
    - status update to NO_SHOW
    - payment status adjustment to FORFEITED

This correction ensured requirement traceability.

Disagreement 3 – Payment-Service Positioning

AI modeled PaymentService as internal.

Problem:

- Req10 describes support for multiple payment systems.
- UML requires external financial systems to be actors.

Resolution:

- PaymentService modeled as external in sequence diagrams.
- Payment class remains internal domain class.

Task 3 – Verifying AI Responses

Verification was done through:

1. Requirement Traceability Check
   Every Req1–Req10 was mapped to:
   - A use case
   - A class or attribute
   - A sequence interaction
2. UML Standard Review
   - Correct use of «include» vs «extend»
   - Proper multiplicity annotations
   - Correct inheritance usage
   - Enumerations instead of separate subclasses where appropriate
3. Manual Logic Validation
   Example:
   AI suggested Reservation “includes” ExtendReservation.
   However, extension is conditional, so we corrected it to «extend».

Example of Technical Verification

AI suggestion:

- One Reservation has exactly one Payment.

Verification:

- Deposit + potential final payment adjustment must be supported.
- Final design allows Payment to track full transaction lifecycle via status.

Outcome:
Modified but concept retained.

Task 4 – Application of AI-Suggested Content

One concrete example of AI-suggested content that was applied directly to Part II (Class Diagram Design) came from an exploratory interaction with DeepSeek.

Original Prompt

> "Give me a class diagram for these requirements:
> ● Req1: All users must register with a unique email and a strong password…
> ● Req2: Only the head lab coordinator can auto-generate lab manager accounts…
> ● Req3: Registered users can reserve available equipment…
> ● Req4: Users must pay one hour's fee as a deposit…
> ● Req5: Sensors track equipment usage…
> ● Req6: Managers can add, enable, or disable equipment…
> ● Req7: Each equipment item has a unique ID…
> ● Req8: Users must provide ID or certification number…
> ● Req9: Users can extend reservation…
> ● Req10: Supports credit, debit, institutional accounts, and research grants."

AI Response (Quoted)

DeepSeek responded with the following structural recommendations:

> "User Inheritance: HeadLabCoordinator and LabManager inherit from User, with the head coordinator having special privileges for account generation."
>
> "Enumerations: Used for fixed sets of values like user types, equipment status, payment methods, etc."
>
> "Reservation-Payment Link: Each reservation has one payment, handling the deposit logic (Req4)."
>
> "Equipment-Sensor Connection: Equipment is monitored by sensors that track usage and send updates (Req5)."
>
> "Usage Tracking: UsageRecord captures detailed sensor data about equipment operation."
>
> "Payment Flexibility: Payment class supports multiple payment methods as specified in Req10."

Final Reflection

AI tools were valuable for:

- Actor identification
- Logical flow suggestions
- Class brainstorming
- Requirement coverage validation

However, multiple corrections were necessary, particularly regarding:

- Actor abstraction
- Deposit enforcement logic
- UML correctness
- Inheritance simplification

The final design reflects human refinement rather than direct AI output reproduction.

Credits:

Part 1:

Task 1: Seyed Parsa Shoushtari (219615574)

Task 2: Doluwamu Olubiremi (218801563)

Task 3: Sadia Jahan Shifa (219302744)

Part 2: Sarah Asghar (219428127)

Part 3: Nabeela Ansari (219730662)